BLDC Motor Controller Development for the PNW Shell Eco-marathon Vehicle

Honors Project Report (ECE 36201 Stacked Course Contract)

Student: Samudra Gargo Bhattacharya
Department: Electrical and Computer Engineering, Purdue University Northwest
Location: Hammond, Indiana

Professor: Dr. David Kozel(ECE 36201)

ABSTRACT

The Shell Eco-marathon competition is driven entirely by efficiency, making the BLDC motor controller a central factor in vehicle performance. Building on my prior involvement in the team's controller development (including significant work on EasyController3 last year), this honors project extends ECE 36201 microcontroller concepts into a real embedded power-electronics system. The work focused on controller architecture planning, microcontroller selection aligned with course hardware (STM32 Nucleo), firmware strategy exploration using the VESC ecosystem to enable FOC without implementing the full algorithm stack from scratch, and risk mitigation through procurement and planning of a backup controller (EasyController2) funded through URD. Weekly technical meetings with the senior design team guided decisions on motor sizing, gate driver options, and system interfaces such as CAN. This semester's outcome is a research and design foundation, preliminary subsystem plans, and a clear execution roadmap for next semester, where PCB completion, board fabrication, assembly, and vehicle integration will occur.

1. PROJECT OBJECTIVES

1.1 Original Project Objectives

The initial direction of my honors work was to contribute directly to the creation of a custom BLDC motor controller for the PNW Shell Eco-marathon vehicle, using a microcontroller-based architecture that reflects and extends the core ECE 36201 topics (PWM generation, timers, interrupts, ADC interfacing, and peripheral coordination). The intention was to work toward an eventual competition-ready controller capable of efficient torque production and telemetry.

1.1.1 Constraints / Risks

Early planning identified key constraints that shaped the project trajectory:

- **FOC complexity risk:** Writing a stable, efficient FOC implementation from scratch is a major multi-semester effort.
- **Competition timeline risk:** A single "new controller" path is fragile if bring-up slips.
- **Hardware dependency risk:** Gate driver choice, power stage validation, and sensing design are all potential bottlenecks.

1.2 Modified Project Objectives

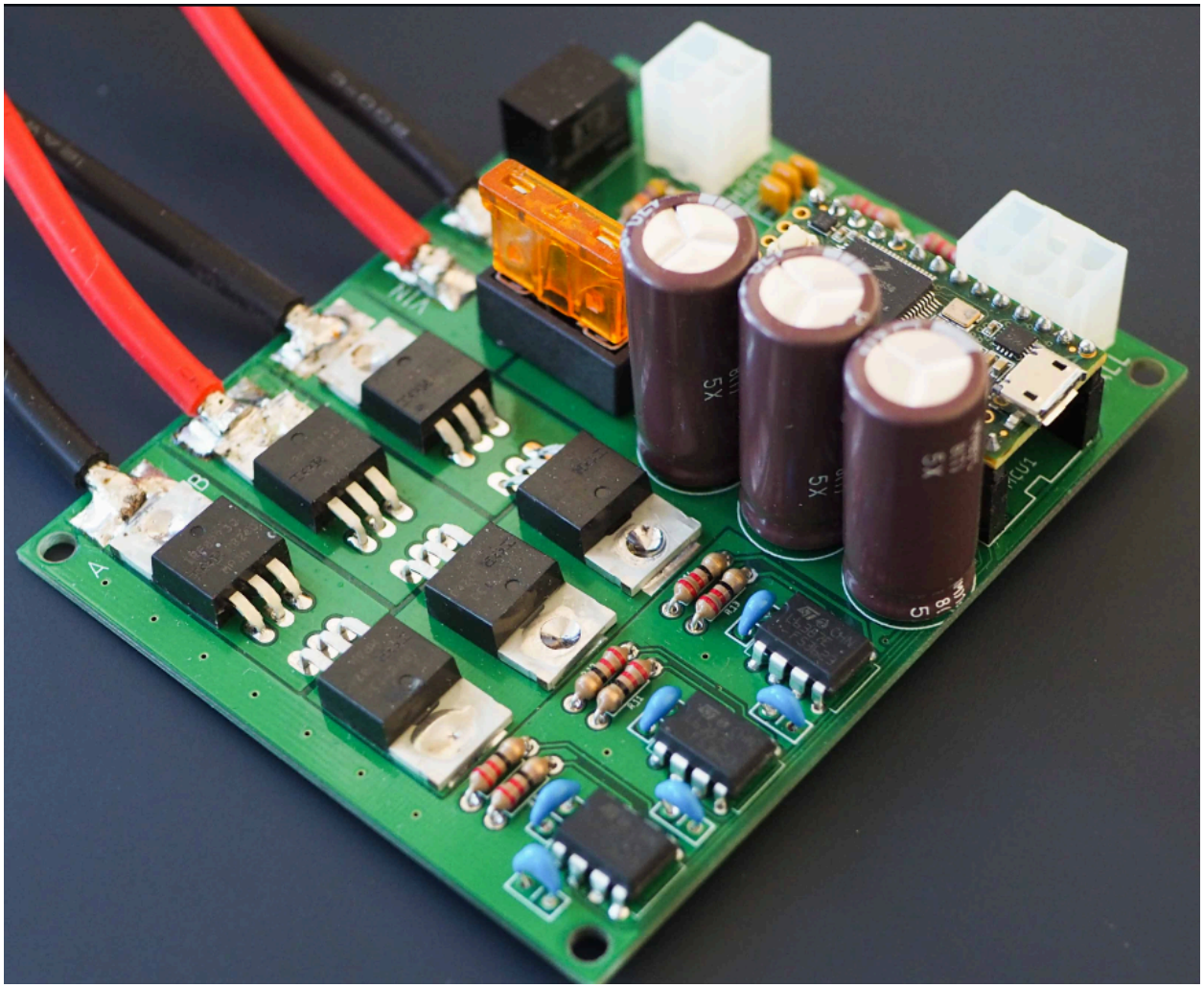
To increase the probability of a working system within the competition schedule, objectives were modified to prioritize **de-risked implementation paths** and tangible subsystem progress rather than overly ambitious “everything from scratch” claims.

1.2.1 De-risking via proven firmware (VESC ecosystem)

Rather than spending the semester attempting to implement FOC math/control loops from nothing, the project shifted toward using **VESC software/firmware** as the control framework. This enables advanced control (including FOC capability) while focusing the team’s effort on hardware design, integration, and porting/compatibility work.

1.2.2 Backup controller strategy (EasyController2)

A backup plan was formalized: I used **URD funds** to purchase components to build an **EasyController2** backup motor controller, ensuring the team has a fallback option if the new VESC-based board is delayed.



1.3 Finalized Project Objectives

By the end of this semester, the finalized objectives became:

1. Stay embedded in the senior design workflow through weekly meetings and design decisions.

2. Support motor selection and power sizing through simulation and requirements discussion.

3. Advance the controller architecture path: **VESC-based control + STM32 direction**.

4. Begin gate driver evaluation through ordered parts + planned breadboard testing.

5. Produce documentation and diagrams (CAN + gate driver block-level circuits) that feed directly into next semester's PCB completion.

2. TECHNICAL DESCRIPTION

2.1 Shell Eco-marathon Vehicle Context

Shell Eco-marathon is an efficiency competition where the propulsion system is evaluated not only by whether it works, but by how little energy it consumes to achieve the required driving profile. In this context, the motor controller is not a generic "ESC"; it directly impacts:

- Conduction losses (current magnitude and waveform)

- Switching losses (PWM frequency, deadtime, gate driver behavior)

- Torque ripple (affects mechanical efficiency and drivability)

- Telemetry quality (ability to measure and improve)

2.1.1 Power + efficiency constraints

Through team discussion and simulation, a decision was made to target a **1000 W motor** based on expected operating requirements and vehicle needs.

We used simulink to chose this motor:

20 inch 48V 1000W rear electric hub motor

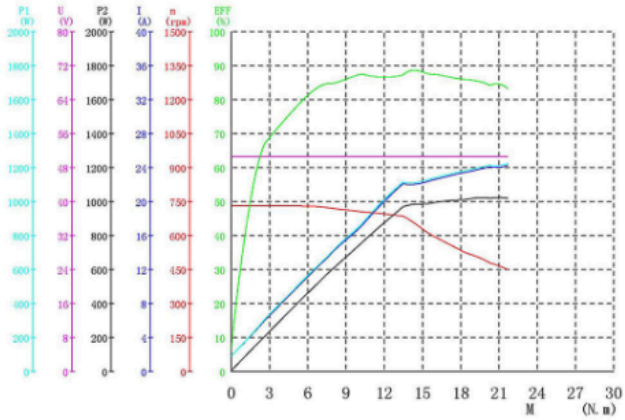
Datasheet:

电机测试报告

客户名称：Leaf Bike　　电机型号：48V / 52V 1000W

测试人员：Leaf Motor　　测试日期：Oct 27, 2020

****



项　目	U (V)	I (A)	P1 (W)	M (N.m)	n (rpm)	P2 (W)	EFF (%)
不加载点	50.55	1.736	87.79	0.06	731.0	4.93	5.6
最高效率点	50.46	21.72	1096	14.43	656.5	992.4	90.5
最大输出功率点	50.45	23.88	1205	21.25	462.3	1028	85.3
最大转矩点	50.45	24.23	1222	21.72	446.9	1016	83.1
结束	50.45	24.23	1222	21.72	446.9	1016	83.1

NO:	U (V)	I (A)	P1 (W)	M (N.m)	n (rpm)	P2 (W)	EFF (%)
1	50.55	1.736	87.79	0.06	731.0	4.93	5.6
2	50.54	2.043	103.3	0.16	728.9	12.22	11.8
3	50.54	3.825	193.3	1.46	730.3	111.8	57.8
4	50.53	6.091	307.7	2.71	729.2	207.0	67.2
5	50.50	10.86	548.6	5.84	729.9	446.7	81.4
6	50.50	12.14	613.1	6.68	727.6	508.9	83.9
7	50.49	13.36	674.9	7.57	721.0	571.5	84.6
8	50.49	14.84	749.3	8.43	715.9	632.4	84.3
9	50.48	15.79	797.6	9.34	709.8	694.5	87.0
10	50.48	17.19	868.1	10.28	705.0	758.8	87.4
11	50.47	19.04	961.4	11.37	697.8	831.1	86.4
12	50.46	20.65	1042	12.41	691.8	899.0	86.2
13	50.46	22.03	1111	13.49	685.8	968.7	87.1
14	50.46	21.72	1096	14.43	656.5	992.4	90.5
15	50.46	22.27	1123	15.22	613.1	977.1	86.9
16	50.46	22.51	1136	15.96	593.3	991.8	87.3
17	50.45	22.87	1154	16.82	569.7	1003	86.9
18	50.45	23.19	1170	17.53	550.1	1009	86.2
19	50.45	23.32	1176	18.26	528.1	1010	85.8
20	50.45	23.51	1186	18.96	512.1	1017	85.7
21	50.45	23.83	1202	19.65	498.9	1026	85.3
22	50.45	24.04	1213	20.33	479.2	1020	84.1
23	50.45	23.88	1205	21.25	462.3	1028	85.3
24	50.45	24.23	1222	21.72	446.9	1016	83.1

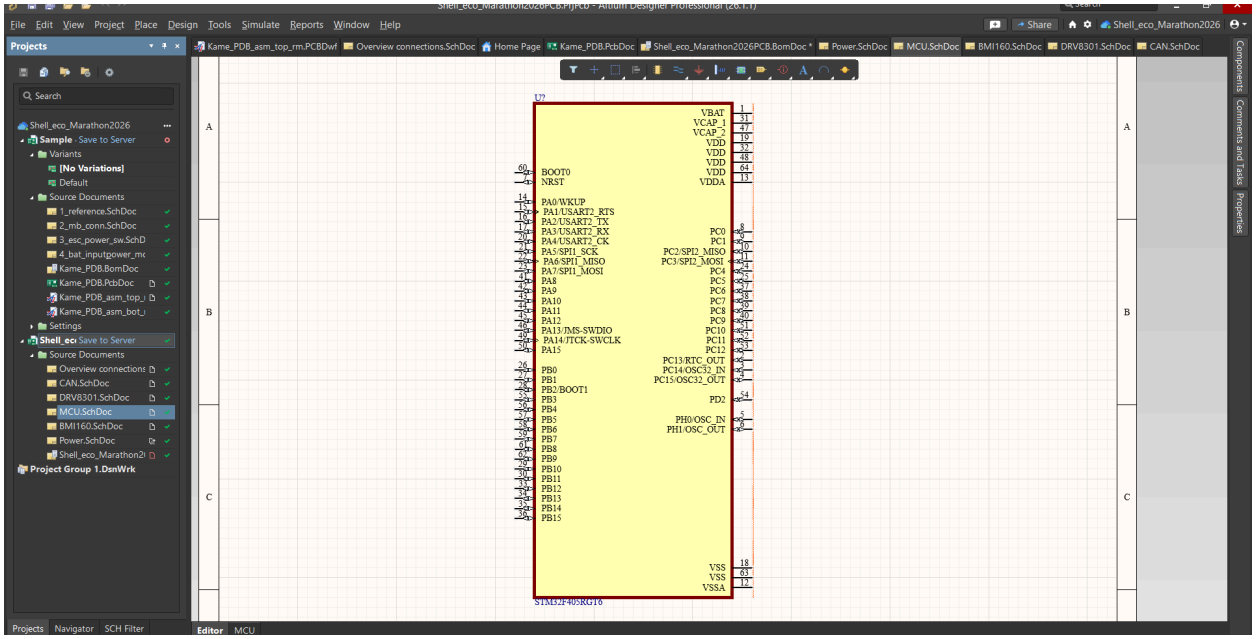
2.2 BLDC Motor Controller Overview

A BLDC controller converts battery DC into controlled 3-phase waveforms using:

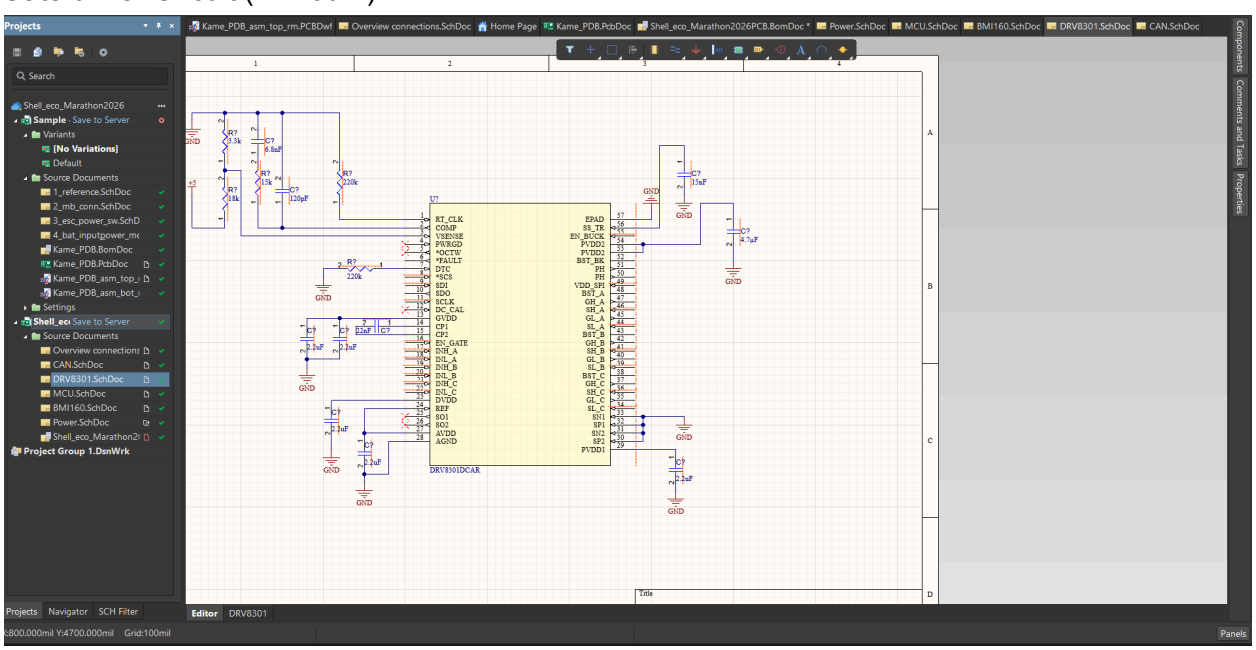
- A 3-phase inverter (6 MOSFETs total)

- A gate driver stage (to switch MOSFETs correctly and safely)
- A microcontroller (PWM timing, ADC sampling, control logic, protections)
- Feedback sensing (current/voltage/temp/position as required)
- Vehicle comms (CAN)

STM32F405:(In Altium)



Gate driver Circuit:(in Altium)



2.2.1 Trapezoidal commutation (backup controller focus)

Trapezoidal (six-step) commutation is simpler and practical for a fallback controller: it produces motor rotation using discrete commutation states and is far easier to implement and debug than full FOC. Next semester, the EasyController2 backup path will be programmed with trapezoidal commutation to ensure a functional baseline.

2.2.2 Field-Oriented Control (FOC) (primary performance path)

FOC enables smoother torque and often improved efficiency by regulating motor currents in a rotating reference frame (Id/Iq). However, implementing it robustly from scratch is high-risk within one semester. For that reason, the project strategy is to leverage the VESC ecosystem's control stack rather than "reinvent it badly."

2.2.3 Microcontroller platform (STM32)

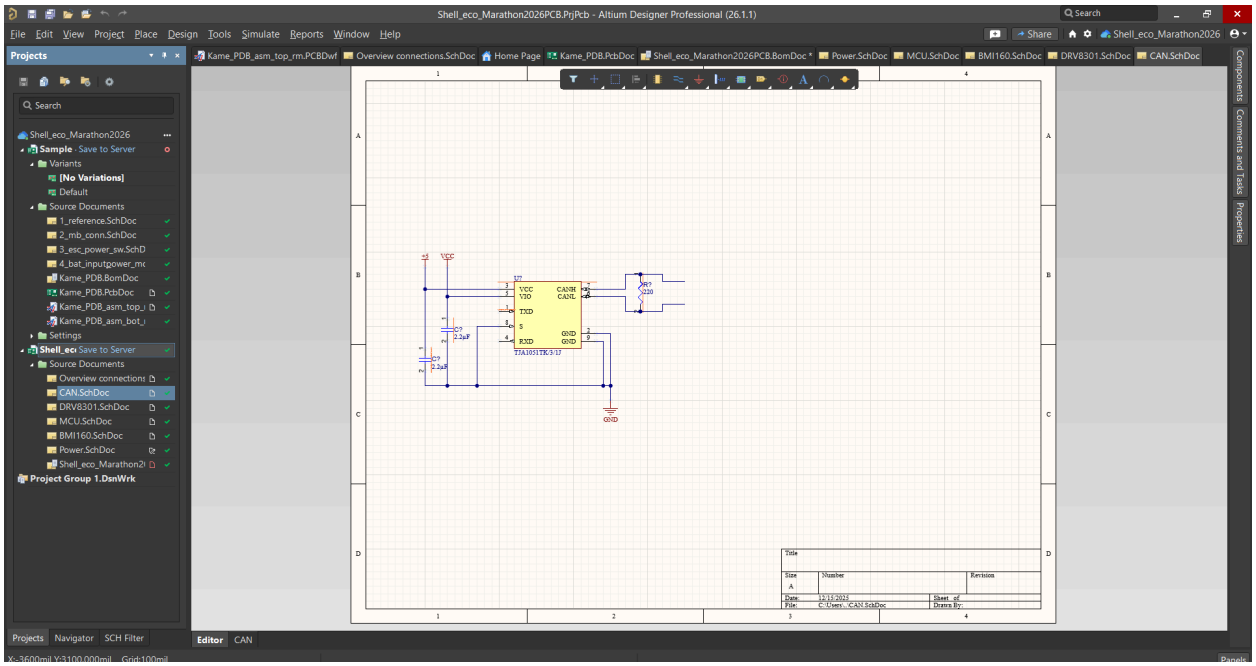
ECE 36201 uses an **STM32 Nucleo** platform, which made STM32 a logical anchor for this honors project. This work is "one level above class" because instead of PWM driving an LED or a simple peripheral demo, PWM and ADC concepts are being planned for a high-power inverter system where timing alignment, protection logic, and noise robustness are real engineering constraints.

2.2.4 Gate driver options + evaluation

The team considered multiple gate driver options (to be narrowed based on switching performance, protections, ease of use, and availability). I ordered gate driver ICs and **test sockets** to enable rapid prototyping and evaluation on a breadboard test setup before final PCB commitment.

2.2.5 CAN + system interfaces

CAN is a major interface for vehicle integration and telemetry. This semester produced early circuit diagram work for CAN and gate driver blocks as a foundation for next semester's PCB completion.



3. METHODS USED

3.1 Weekly collaboration and planning

I met with the senior design team **every week** to align on system architecture decisions and execute the planning work needed for a multi-semester controller build. These meetings were used to:

- review options (microcontroller, firmware direction, gate drivers)
- coordinate tasks between EE and ME groups
- track risks and timeline (especially around PCB deadlines)

3.2 Procurement and prototyping

Using URD funding, I purchased components to support the backup controller strategy (EasyController2 build) and to enable gate driver evaluation without waiting for a full custom PCB. This approach improves iteration speed and reduces schedule risk.

3.3 Simulation and power sizing

The team ran simulations and calculations to estimate motor power requirements and selected a **1000 W** motor accordingly. I contributed to this process through discussion, interpretation, and integration of results into the controller planning.

3.4 Firmware exploration and porting direction

To avoid wasting time on fragile, from-scratch FOC, the team used VESC software as the control planning foundation. I collaborated with a senior teammate to learn how to program a Teensy and then shifted focus toward the current direction: getting **VESC firmware running on STM32**, which is the platform aligned with course tools and the controller hardware plan.

3.5 PCB Design Tool Learning

We learned Altium PCB designer, as it is an industry standard and allows multiple people to collaborate.

4. CONCLUSION

This semester established real foundations for a competition-relevant motor controller effort:

- Embedded weekly in senior design planning and execution
- Formalized a backup controller path (EasyController2) and procured parts
- Supported motor selection and settled on a 1000 W motor direction
- Built early hardware planning artifacts (CAN + gate driver diagrams)
- Started gate driver evaluation workflow via ordered ICs + test sockets
- Advanced firmware strategy toward VESC-based FOC enablement and STM32 alignment

4.2 Current limitations / risks

If you’re honest about engineering (and you should be), these are the real risks:

- Porting/hosting VESC firmware on STM32 is non-trivial and can become a time sink.

- Gate driver choice can make or break reliability; testing must be real, not theoretical.
- PCB bring-up always reveals issues—without a backup controller, the schedule can collapse (hence EasyController2).

4.3 Future work (next semester execution plan)

Next semester will be build-and-validate:

1. Finish the VESC-based board design (schematic + layout) and send to fabrication.
2. Assemble/solder and perform hardware bring-up (power stage + drivers + sensing).
3. Complete EasyController2 backup assembly and implement trapezoidal commutation.
4. Open the motor and confirm pole count / pole pairs to support correct FOC configuration.
5. Integrate CAN and validate telemetry.
6. Vehicle integration + testing + Shell Eco-marathon competition execution.

References:

https://github.com/pgrady3/EasyController2

https://vesc-project.com/

https://www.pmdcorp.com/resources/type/articles/get/field-oriented-control-foc-a-deep-dive-article?hs_amp=true

https://www.youtube.com/playlist?list=PLgRKdL8313Vh5pNHsYSNzR85VyOE-iC3h&si=mFM1xdYvIS9I6TpS